

黑名单因子查询服务（blk）• API 接口文档与使用手册

面向接入方（客户）技术与运维人员的对外接口说明。

版本：blk（黑名单因子） | **通信：**HTTP + JSON | **编码：**UTF-8

说明：本服务采用统一的请求信封与 MD5 加签方式，响应分为 `head`（网关头部）与 `body`（业务结果）两部分。其余内部处理细节调用方无需关心。

关键特性：黑名单因子的查询结果是一个结构化对象（命中等级、命中场景等），本服务将其原样序列化为 JSON 字符串置于 `body.result.range`，由调用方自行解析（见 3.1.5 `result.range` 结构说明）。

计费提示：本服务按**有效查询次数**计费——查得数据（001）与查无结果（999）**均计费**，详见 五、计费说明。

一、接入必读

1.1 适用范围

本文档适用于接入本平台「黑名单因子查询服务（blk）」的第三方产品技术开发人员与日常运维人员。

1.2 接入须知

- 正式访问域名在接入时由我方商务提供。
- 接入前需先申请开通账户，由我方分配 `appKey` 与 `appSecret`（加签密钥）。

1.3 接口说明

项目	说明
请求方式	POST（计费查询数查询为 GET）
通信协议	HTTP
数据格式	请求体与响应体均为 JSON
字符编码	UTF-8
超时时间	4 秒（建议客户端读超时 $\geq$ 5 秒）
HTTP 状态码	恒为 200；业务结果与错误均通过响应体的 <code>head.errorCode</code> / <code>body.code</code> 表达
签名	调用时需对 <code>body</code> 中的业务参数 + 我方分配的 <code>appSecret</code> 进行 MD5 加签，详见 二、鉴权与加签

1.4 环境说明

- 正式环境：**使用正式账户，调用已开通接口，**按有效查询次数计费**（见 五、计费说明）。

- **测试环境**：使用测试账户，返回挡板/联调数据。
- 正式账户仅适用于正式环境，测试账户仅适用于测试环境。

二、鉴权与加签

所有业务接口共用同一套请求信封与鉴权方式。

2.1 请求信封（顶层参数）

参数	示例	类型	必填	说明
appKey	y8909b1k	String	是	我方分配给客户的公开标识
sign	0528999dd55c025b8f36fc72dceb1f63	String	是	对 body 业务参数的 MD5 签名（见 2.3）
encryptionType	1	int	否	参数加密类型，1 = 明文（默认）
body	{...}	Object	是	业务请求体，见各接口定义

注意：本服务不需要 `apiKey` 参数（产品由请求路径区分）。`appKey` / `sign` / `encryptionType` 不参与签名计算。

调用方传入的 `body` 字段一律为**明文**（手机号/身份证号/姓名）。隐私保护所需的 PII 摘要（MD5）由本服务内部完成，调用方无需做任何加密。

2.2 鉴权校验顺序

网关按以下顺序校验，任一失败立即返回对应 `head.errorCode`（直接拒绝，不计费）：

1. `appKey` 是否存在 → 否则 505001
2. `appKey` 是否匹配到账户 → 否则 505004
3. 账户是否有效（启用且在有效期内）→ 否则 505007
4. 签名是否正确 → 否则 505002

2.3 加签方式

1. 取出 `body` 中所有**非空**的业务参数（不含文件/字节流类型，不含值为空的参数）。
2. 按参数名（key）的 **ASCII 升序**排序；首字符相同则依次比较后续字符。
3. 将排序后的参数按 `参数名 参数值` 直接拼接，最后追加 `appSecret`，得到待签名串。
4. 对待签名串做 **MD5**，取 **32 位小写十六进制**字符串，赋值给 `sign`。

示例： `body = { "mobile": "13809091009", "idCard": "330129199109094312", "name": "张三" },`
`appSecret = "<你的密钥>"`。

按 ASCII 升序排序后 key 顺序为 `idCard` → `mobile` → `name`，待签名串为：

```
idCard330129199109094312mobile13809091009name张三<你的密钥>
```

`sign = MD5(待签名串)` 的小写十六进制值。

提示：拼接顺序由 key 的 ASCII 决定，请勿写死字段顺序；新增字段时排序会自动变化。

2.4 加签代码示例

Java

```
public static String sign(Map<String, String> bodyParams, String appSecret) throws Exception {
    StringBuilder sb = new StringBuilder();
    List<String> keys = new ArrayList<>(bodyParams.keySet());
    Collections.sort(keys); // ASCII 升序
    for (String k : keys) {
        String v = bodyParams.get(k);
        if (v == null || v.isEmpty()) continue; // 剔除空值
        sb.append(k).append(v);
    }
    sb.append(appSecret);
    MessageDigest md = MessageDigest.getInstance("MD5");
    byte[] digest = md.digest(sb.toString().getBytes(StandardCharsets.UTF_8));
    StringBuilder hex = new StringBuilder();
    for (byte b : digest) hex.append(String.format("%02x", b));
    return hex.toString();
}
```

Python

```
import hashlib

def sign(body_params: dict, app_secret: str) -> str:
    parts = []
    for k in sorted(body_params.keys()):          # ASCII 升序
        v = body_params[k]
        if v is None or v == "":
            continue                             # 剔除空值
        parts.append(f"{k}{v}")
    raw = "".join(parts) + app_secret
    return hashlib.md5(raw.encode("utf-8")).hexdigest() # 小写 hex
```

Go

```
func sign(body map[string]string, appSecret string) string {
    keys := make([]string, 0, len(body))
    for k, v := range body {
        if v != "" { // 剔除空值
            keys = append(keys, k)
        }
    }
    sort.Strings(keys) // ASCII 升序
    var sb strings.Builder
    for _, k := range keys {
        sb.WriteString(k)
        sb.WriteString(body[k])
    }
    sb.WriteString(appSecret)
    sum := md5.Sum([]byte(sb.String()))
    return hex.EncodeToString(sum[:]) // 小写 hex
}
```

三、接口列表

3.1 黑名单因子查询 (blk)

项目	内容
路径	POST /v1/openapi/zlx/querySrmxBLK
完整地址	http://www.aiszcloud.cn:8080/v1/openapi/zlx/querySrmxBLK
鉴权	appKey + MD5 签名 (见第二章)

3.1.1 请求 body 参数

参数	示例	类型	必填	说明
mobile	13809091009	String	是	手机号 (11 位, 1 开头), 明文传入
idCard	330129199109094312	String	是	身份证号 (18 位, 末位可为 x), 明文传入
name	张三	String	是	姓名, 明文传入

参数格式非法 (手机号/身份证号不符) 将返回 head.errorCode = 505062 , 直接拒绝, 不计费。

3.1.2 请求示例

```
{
  "encryptionType": 1,
  "appKey": "y8909blk",
  "sign": "0528999dd55c025b8f36fc72dceb1f63",
  "body": {
    "mobile": "13809091009",
    "idCard": "330129199109094312",
    "name": "张三"
  }
}
```

3.1.3 响应结构

响应分为 head（网关头部）与 body（业务结果）两部分：

head 字段：

参数	示例	类型	说明
errorCode	0	String	网关返回码。0 = 成功（含查得/查无）；非 0 = 网关级错误，此时无 body
errorMsg	success	String	返回描述
logId	a1b2c3...	String	全链路追踪 ID，排障/对账时请提供
time	128	Number	服务处理耗时（毫秒）
timestamp	1718456789012	Number	响应时间戳（毫秒）

body 字段（仅 head.errorCode = 0 时返回）：

参数	示例	类型	说明
code	001	String	业务结果码。001 = 查得数据【计费】；999 = 查无结果【计费】
msg	成功	String	业务描述
reqid	1kf9x2...	String	本次请求流水号
uid	lgt1721198...	String	交易流水号（对账用）
result	{...}	Object	业务内容，仅 code = 001 时存在
result.range	"{"whether_hit":1,...}"	String	黑名单因子结果的 JSON 字符串，调用方需 JSON.parse 后使用，结构见 3.1.5

3.1.4 响应示例

① 查得数据 (计费)

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d4", "time": 132, "tr
  "body": {
    "code": "001",
    "msg": "成功",
    "reqid": "1kf9x2ab",
    "uid": "lgt17211982299952",
    "result": {
      "range": "{\\"whether_hit\\":1,\\"hit_grade\\":3,\\"hit_type\\":[{\\"scene\\":\\"P1\\",\\"m1\\":0
    }
  }
}
```

② 查无结果 (计费)

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d5", "time": 96, "timeo
  "body": {
    "code": "999",
    "msg": "未查得",
    "reqid": "1kf9x2ac",
    "uid": "8a1d6b22"
  }
}
```

999 表示本次查询已完成、库中确无该主体的命中记录——这是一个**有效的查询结论**，与 001 同样计费（见 五、计费说明）。

③ 网关级错误 (无 body)

```
{
  "head": { "errorCode": "505002", "errorMsg": "账号信息异常", "logId": "a1b2c3d6", "time": 1678901234 }
}
```

3.1.5 result.range 结构说明

`result.range` 是黑名单因子结果对象序列化后的 JSON 字符串（完整结果原样返回，未做裁剪）。调用方应先对该字符串做一次 `JSON.parse` / 反序列化，再读取以下字段：

字段	类型	说明
<code>whether_hit</code>	int	是否命中：0 = 未命中，1 = 命中
<code>hit_grade</code>	int	命中等级，取值 0 – 5（数值越大风险越高）
<code>hit_type</code>	Array	命中场景明细数组，每个元素对应一个风险场景（P1–P8 共 8 类）
<code>hit_type[].scene</code>	String	场景标识（P1–P8）
<code>hit_type[].m1</code>	int	近 1 个月在该场景的命中次数
<code>hit_type[].m3</code>	int	近 3 个月在该场景的命中次数
<code>hit_type[].m6</code>	int	近 6 个月在该场景的命中次数

说明：`result.range` 的具体字段可能随数据能力升级扩展，本服务保证完整透出。建议调用方按「存在即解析、缺失即忽略」的方式做容错处理，避免因后续新增字段导致解析失败。

解析示例（JavaScript）：

```
const range = JSON.parse(resp.body.result.range);
if (range.whether_hit === 1) {
  console.log("命中等级:", range.hit_grade);
  range.hit_type.forEach(t => console.log(t.scene, t.m1, t.m3, t.m6));
}
```

3.2 计费查询数查询（扩展接口）

查询本账户累计**计费查询**的次数，用于客户侧自助监控与对账。不返回额度上限/剩余量（本服务无额度限制）。

项目	内容
路径	<code>GET /v1/openapi/zlx/quotaBLK</code>
鉴权	与主接口一致（请求体中携带 <code>appKey</code> + <code>sign</code> 信封； <code>body</code> 可为 <code>{}</code> ，此时 <code>sign</code> = <code>MD5(appSecret)</code> ）

响应示例

```
{
  "errorCode": "0",
  "errorMsg": "success",
  "status": "ACTIVE",
  "serviceUsed": 1280,
  "totalCalls": 1560
}
```

参数	说明
status	账户状态 (ACTIVE/SUSPENDED 等)
serviceUsed	累计 计费 查询次数 (code = 001 与 code = 999 均计入)
totalCalls	累计有效查询次数 (含查得与查无; 不含被网关拦截的请求)

说明: 无任何额度上限拦截, 仅做计费次数统计。本服务对查得与查无同等计费, 故 serviceUsed 与 totalCalls 在正常情况下一致; 对账请以 serviceUsed 为准。

3.3 健康检查

项目	内容
路径	GET /healthz
鉴权	无
响应	HTTP 200, 纯文本 ok

四、返回码说明

4.1 网关返回码 head.errorCode

errorCode	含义	典型原因
0	成功	调用成功（业务结果见 body.code ）
505001	appKey 异常	缺少或非法 appKey
505004	账户信息不存在	appKey 未匹配到账户
505007	服务尚未开通	账户停用 / 过期 / 未开通
505002	账号信息异常	签名校验失败
505003	产品编号异常	保留
505062	数据请求异常	参数非法 / 服务处理异常 / 超时未决 / 系统错误（默认错误码）

服务内部处理异常均统一返回 505062，**不计费**，并经异步复查兜底。

4.2 业务结果码 body.code（仅 errorCode = 0 时）

code	含义	是否计费
001	查得数据	计费
999	查无结果	计费

注：本服务按**有效查询次数**计费。只要本次查询产出了确定结论——无论是查得命中数据（001）还是确认无命中记录（999）——均计费一次。仅当返回 head.errorCode 非 0（未能产出结论）时不计费。

五、计费说明

- body.code = 001（查得数据）与 body.code = 999（查无结果）均计入计费查询次数并对客户计费。
本服务按**有效查询次数**计费：查无结果同样消耗一次查询并产出确定结论。
- 网关级错误（head.errorCode 非 0：鉴权失败、参数非法、系统异常等）**一律不计费**。
- 计费以最终落库的台账为准，超时未决请求会经异步复查/对账裁定状态，不会重复计费。
- 计费次数可随时通过 计费查询数查询（3.2）接口自助核对。

六、使用手册（接入与最佳实践）

6.1 接入流程

1. 向商务申请账户，获取 `appKey`、`appSecret`、正式/测试域名。
2. 按第二章实现加签，先在测试环境联调，再切正式环境。
3. 上线后通过计费查询数查询接口（3.2）监控调用量与计费次数。

6.2 幂等与重试

- 客户端建议为每笔查询设置合理超时（ ≥ 5 秒）。
- 收到网络超时/无响应时**可安全重试**：网关基于内部流水号做幂等，不会因重试重复计费。
- 请勿对已明确返回 `head.errorCode` 的请求做无差别重试（如参数错误 `505062`、鉴权错误 `505001/505002/505004`），应先修正再发起。

6.3 错误处理建议

现象	排查方向
505001 / 505004	检查 <code>appKey</code> 是否正确、是否用错环境账户
505002	检查签名算法（排序/空值剔除/UTF-8/小写 hex）
505007	联系商务确认账户状态与有效期
505062	检查 <code>mobile / idCard / name</code> 是否齐全合法；若入参正常仍持续出现，记录 <code>logId</code> 联系我方

任何异常排查请一并提供响应中的 `head.logId`，便于我方全链路定位。

6.4 联调自检清单

- ☐ 域名、`appKey`、`appSecret`、环境匹配无误
- ☐ 待签名串严格按 ASCII 升序拼接、剔除空值、UTF-8、MD5 小写
- ☐ `mobile / idCard / name` 均以**明文**传入（PII 摘要由网关内部处理）
- ☐ 能正确解析 `head.errorCode` 与 `body.code` 两级状态
- ☐ **已知悉 999 查无结果同样计费**，业务侧计费预估按有效查询次数而非命中数测算
- ☐ 已实现对 `result.range` 字符串的二次 `JSON.parse` 解析与字段容错
- ☐ 已实现超时重试（依赖幂等，不重复计费）

附录：术语表

术语	说明
appKey	公开账户标识，随请求明文传输
appSecret	加签密钥，仅本地保存用于计算 sign ，切勿泄露或随请求传输
logId	全链路追踪 ID (= head.logId) ，排障/对账唯一凭据
range	黑名单因子结果的 JSON 字符串，需二次解析（结构见 3.1.5）
有效查询	产出了确定结论的查询（ body.code = 001 或 999 ） ，即本服务的计费单位